

Objetivo

Desarrollar una API utilizando Django y Django REST Framework (DRF) que implemente autenticación mediante JSON Web Tokens (JWT) y permita a los usuarios gestionar sus dispositivos en el contexto de múltiples plataformas.

La prueba está pensada para completarse en un máximo de 2 - 3 días.

⚠ Nota importante: Se permite y valora el uso de herramientas de IA durante el desarrollo. En la entrevista posterior se evaluará tu capacidad de explicar, defender y modificar cada decisión técnica del código entregado.

Descripción general

El sistema debe permitir la existencia de varias plataformas (por ejemplo, "Plataforma A", "Plataforma B", etc.).

Un mismo usuario podrá registrarse en distintas plataformas utilizando el mismo email, y deberá poder iniciar sesión de manera independiente en cada una de ellas.

Una vez autenticado, el usuario podrá acceder a un endpoint protegido que devuelva los dispositivos asociados a su cuenta en la plataforma actual.

Cada plataforma puede tener reglas de negocio propias configuradas (ej: número máximo de dispositivos permitidos). El sistema debe soportar estas reglas de forma configurable sin modificar código.

Requisitos funcionales

- Autenticación por JWT
 - El inicio de sesión debe realizarse mediante email, contraseña y plataforma.
 - El sistema debe devolver un token JWT válido únicamente para la plataforma especificada.
 - Los endpoints protegidos deben requerir este token para acceder.
- Gestión de plataformas
 - Debe existir un modelo o estructura que represente una plataforma.
 - Cada plataforma puede tener reglas de negocio configurables (ej: max_devices, allow_inactive_login). Diseña la solución que consideres más adecuada.
 - Un mismo usuario puede estar registrado en una o varias plataformas, sin conflicto por email repetido.
- Gestión de dispositivos
 - Cada usuario puede tener varios dispositivos asociados a su cuenta dentro de una plataforma.

- La API debe incluir un endpoint que devuelva solo los dispositivos pertenecientes al usuario autenticado y a la plataforma correspondiente.
- Al registrar un nuevo dispositivo, el sistema debe validar que no se supere el límite configurado para esa plataforma.
- Cada dispositivo debe incluir:
 - Nombre del dispositivo
 - Dirección IP
 - Estado activo o inactivo
 - Fecha de última conexión (last_seen)
- Notificaciones
 - Cuando un usuario se registra en una plataforma, debe dispararse una notificación (puede ser un simple log o print simulado).
 - Diseña el sistema de notificaciones pensando en que en el futuro habrá otros tipos de notificaciones y otros eventos que las disparen. Justifica tu diseño en DECISIONS.md.

Requisitos técnicos

- Lenguaje: Python 3.x
- Frameworks: Django y Django REST Framework
- Autenticación: JWT
- Base de datos: libre elección (SQLite es suficiente)
- El email debe ser el campo de identificación principal del usuario dentro de cada plataforma.

No es necesario desarrollar una interfaz gráfica; basta con endpoints funcionales comprobables mediante Postman, cURL o herramienta similar.

Entregable

El candidato deberá entregar un repositorio (archivo ZIP o enlace Git) que incluya:

- El código fuente del proyecto.
- Un archivo README.md con:

- Instrucciones de instalación y ejecución.
- Pasos para crear plataformas y usuarios de prueba.
- Ejemplo de obtención del token JWT.
- Ejemplo de llamada al endpoint de dispositivos.
- DECISIONS.md (OBLIGATORIO).
 - Al menos 3 decisiones de diseño que tomaste, las alternativas que consideraste y por qué las descartaste.
 - Cómo implementaste las reglas configurables por plataforma y por qué elegiste ese approach.
 - Cómo diseñaste el sistema de notificaciones y qué consideraste para hacerlo extensible.
 - Qué harías diferente si el sistema tuviera 1 millón de usuarios.
 - Qué parte de tu código moverías primero a un microservicio FastAPI y por qué.

Este documento se revisará en detalle en la entrevista técnica. No hay respuestas correctas o incorrectas; lo que se evalúa es tu capacidad de razonar sobre trade-offs.

Criterios de evaluación

- Correcto manejo del inicio de sesión por plataforma.
- Uso del email como identificador dentro de cada plataforma.
- Implementación correcta del sistema de autenticación JWT.
- Implementación de reglas configurables por plataforma.
- Endpoint de dispositivos correctamente filtrado por usuario y plataforma.
- Diseño del sistema de notificaciones y su extensibilidad.
- Limpieza, organización y legibilidad del código.
- Claridad del README y facilidad para ejecutar el proyecto.
- Calidad y claridad del DECISIONS.md — justificación de decisiones de diseño.
- Buenas prácticas de Django y Django REST Framework.
- Pruebas sobre el código del proyecto.

Extras valorables

- Configuración mediante docker compose.
- Uso de Celery para el sistema de notificaciones.
- Validaciones adicionales (por ejemplo, restricción de IPs o control de estado activo del usuario).
- Paginación o filtros en el listado de dispositivos.
- Implementar algún endpoint como microservicio FastAPI independiente.